Software Engineering

- **Software**: a program or set of programs containing instructions that provide desired functionality.
- **Engineering**: the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

一组程序，能提供功能

- **Software engineering**: the branch of engineering that deals with the design, development, testing, and maintenance of software applications, while ensuring that the software to be built is:
  - Correct
  - Consistent
  - On budget
  - On time
  - Within the required requirements.

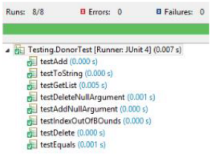
Software Engineering vs. Programming

- **Programming** is a significant part of software engineering
  - Programming is how you generate a new software in the first place
- **Software engineering** is programming integrated over **time, scale, and trade-offs**

Programming

- **Short-term**: your programming code is likely to last for only hours, days, or weeks, not any longer (i.e., decades)
- **No-change**: You probably won't upgrade and maintain your programming code after the assignment deadline ☺
- **Human resources**: programming assignments are done by individuals or 2-3 sized small groups
- **Computing resources**: a single laptop is generally sufficient for programming assignments

- **Correctness**
- **Time (e.g., deadline)**



Software Engineering

- **Long-term**: large software (e.g., Microsoft Office, Google Search) tend to live for decades
- **Adapt-to-change**: to allow for longer life spans, software needs to adapt to new versions of underlying dependencies, OS, hardware, programming language versions, etc.
- **Human resources**: large software is developed and maintained by (large) teams
- **Computing resources**: as organization and users grow, large software needs to scale well with compute, memory, storage, bandwidth resources

- **Software quality**
- **Engineering efforts**
- **Financial costs**
- **Resource costs (e.g., CPU time)**
- **Social impact**
- **...**

Life span of software

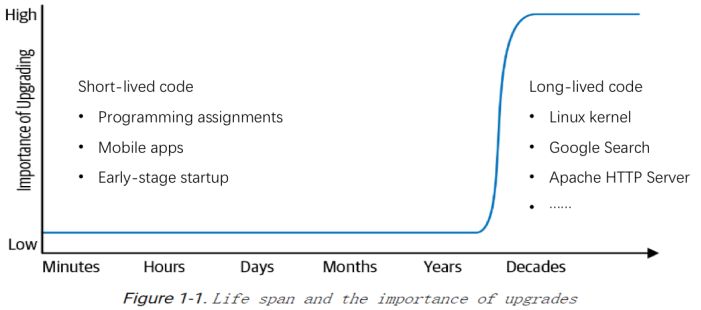


Figure 1-1. Life span and the importance of upgrades

Hyrum's Law

With a sufficient number of users of an API,
it does not matter what you promise in the contract:
all observable behaviors of your system
will be depended on by somebody.

- On the one hand, having more consumers is a good strategy to increase the value that an API generates.
- On the other hand, the more consumers there are, the more likely it is that (because of Hyrum's law) some of them are negatively affected by changes in the API that in theory shouldn't have affected anybody.

在实践中，有的消费者的不应该行为会改变整个系统的设计

```
>>> for i in {"apple", "banana", "carrot", "durian", "eggplant"}: print(i)
...
durian
carrot
apple
eggplant
banana
```

- User 1: assume that the hash iteration ordering is fixed and write some code that depends on the ordering
- User 2: assume that the hash iteration ordering is random, and use it as an inefficient random-number generator
- User 3: assume that the hash iteration ordering is random and never write any code that depend on the ordering.

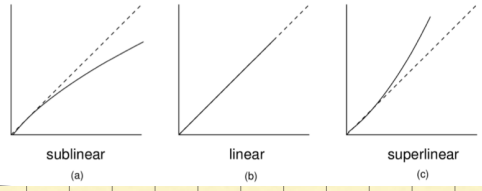
User 1 breaks if the hash ordering becomes randomized

User 2 breaks if the hash ordering becomes fixed

User 4 breaks if the results produced by user 3 are consumed by user 4, whose code depends on the order of the results

Scalability

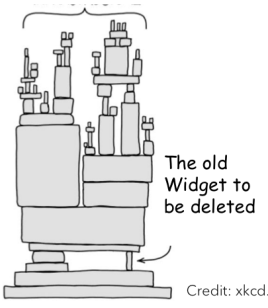
- Your organization's codebase is sustainable when you are able to change all of the things that you ought to change, safely, and can do so for the life of your codebase.
- If costs grow **superlinearly** over time, the operation clearly is **not scalable**.



- X axis: the demand (e.g., codebase)
- Y axis: resources costs
  - How many additional computing resources (e.g., memory, storage) are needed?
  - How many additional human involvement is needed?
  - How long does it take to do a full build?
  - How long does it take to pull a fresh copy of the repository?
  - How much will it cost to upgrade to a new language version?
  - ....

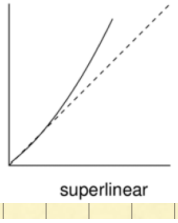
Case Study

All the nice projects that are built



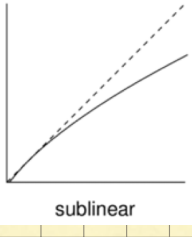
- This type of approach might work in a small software setting, but quickly fails as both the depth and breadth of the dependency graph increases.
- Teams depend on an ever-increasing number of Widgets, and a single build break can affect a growing percentage of the company.

Dependent users/companies quickly grow: not scalable



- A dedicated group of experts in the infrastructure team
  - Either do the update in place, in backward-compatible fashion.
  - Or do the work to migrate their users to new versions
- Having a dedicated group of experts execute the change **scales better** than asking for more maintenance effort from every user

A small group of experts: scales better



General rule of reducing cost : Shift left

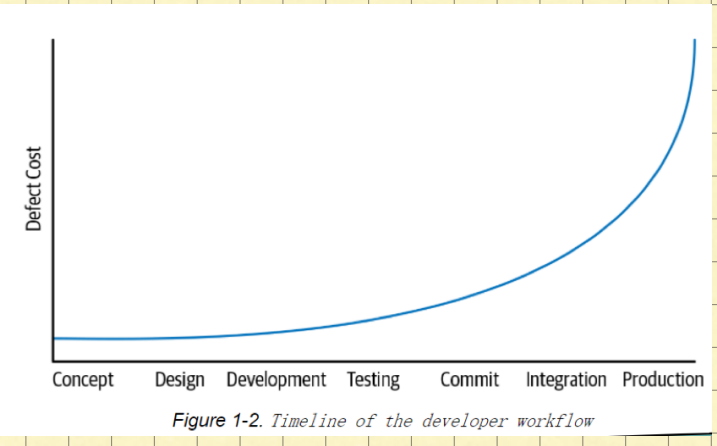


Figure 1-2. Timeline of the developer workflow